

Software Technology (7170) Group Assessment Report

Unit Code : 7170
Semester : 1 2026

Data Structures (u3276740)

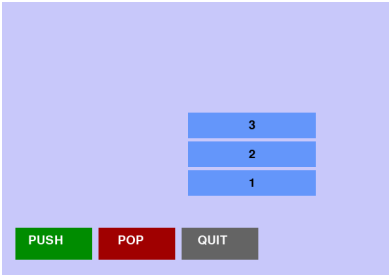
All of the functional code for “*Data Structures*” is implemented in *data_structures_visuliser.py* with the class logic of stacks, queues, linked lists, and binary search tree existing in separate files to keep the classes clean and easily readable.

- **Selection screen**
 - The code of the selection screen is based on code provided in the Week 10 tutorial and computer lab document, it functions by the `main.py` file calling `data_structures_visualiser.py` which on run displays the selection screen and waits to run the visualisers until they are selected.
- **Stack Visualisation**
 - Both the `def stack_visualiser` and `stack` class code is based on code provided in week 3 lecture material as well as the Week 10 tutorial and computer lab document. The only modifications necessary was the addition of animation components for push/pop as well as restructuring the code to function with on-screen buttons instead of keyboard inputs.
- **Queue Visualisation**
 - The queue class code is based on code provided in week 3 lecture material as well as heavily modifying the code of the `def stack_visualiser`. The main modifications required was altering the animations to function from right to left of the screen and redraw the blocks after values have been dequeued.
- **Linked List Visualisation**
 - The `def linked_list_visualiser` requires two functions `def draw_linked_list` to display the list and change colours and `def linked_list_visualiser`, which was initially difficult as the linked list class code provided in week 4 and week 6 tutorial conflicted with efforts to add animations to the visualiser, additionally due to the nature of linked list the visualiser needed the ability for users to input specified values if they needed to. These two issues took the largest amount of time to solve and involved additional research into pygames input boxes as well as experimentation and research of the linked list classes.
- **BST Visualisation**
 - The `bst_visualiser` requires two functions to operate one `def draw_tree` to draw/display the tree and its nodes as well as responsible for changing the

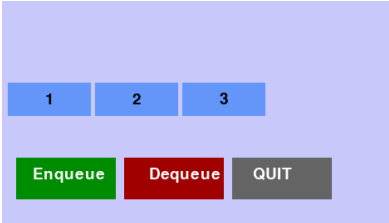
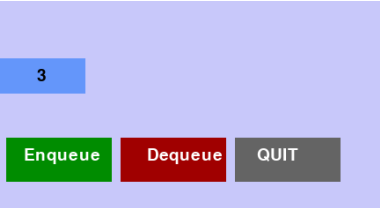
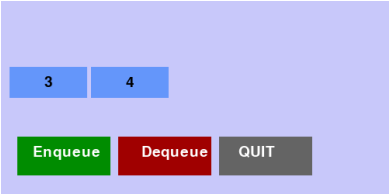
nodes colour, and the `bst_visualiser` to run the visualisers functionality and logic such as buttons and animations. The main issue involved with the `bst_visualiser` was altering the class logic provided in week 7 to allow for the implementation of animations.

Testing demonstration

Stack

Push 3 Items	Pop 2 Items	Push 1 Item
		

Queue

Enqueue 3 Items	Dequeue 2 Items	Enqueue 1 Item
		

Sorting Section (u3305900)

All code for the sorting section is done in the files `sorting.py` and `merge_sort.py`, it was separated for ease of workflow and due to issues I had with the merge sort section. As well as minor edits to `main.py` for linking code to main menu

- **Reset button**
 - Had minor issues originally until i figured out that when pressed a new array needed to be generated
- **The selection screen**

- Designed so once a selection is done it automatically calls the assigned method for the sort design, code is based on the code from the previous sections with edits to how each choice is called.
- **Bubble sort**
 - The bubble sort section was taken directly from the week 11 tutorial work as task2_3 [done in one module called `bubble_sort_visualize`]
- **Selection sort**
 - A variation based on the code from the bubble sort algorithm, no issues with the creation process. [done in one module called `selection_sort_visualize`]
- **Merge sort**
 - Issues with this one lead to it being done in a separate file to make testing easier, tried to do it based on the existing code but it did not work at all with the original set up, so after a while and some research this code was made based on code found at <https://www.geeksforgeeks.org/dsa/sorting-algorithm-visualization-merge-sort/> After a while of fiddling I could not figure out how to make the array match the previous ones but left it as it worked well enough.

Graphs (u3305900)

All code for the section is found in the files `graph_explore.py` and `heaps_visualiser` as well as minor edits to `main.py` to link to the main menu. Separated due to unsure placement of where the heap visualizer until later developments.

- **Selection screen**
 - Same set up and base code as the sorting sections selection screen, no issues.
- **BFS**
 - Code from week 12 tutorial task 3_1
- **DFS**
 - Made with variations on the code from task 3_1
- **Main screen**
 - To make the nodes as a start point work as well as the reset button all of these options are done in the `main()` function and the selected choice of BFS or DFS is done with variables.
- **Heap visualizer**
 - The code for this section is based on the code from week 12 tutorial task3_2, the only issues that took working out with this section was figuring out how to make inserts work with specific values, this code ended up being based off of work done by my groupmate in the data structures section. There is a minor error in this part which is that you need to click twice after entering a value in the box for it to be registered but I could not find a way to fix it.

Puzzle Games (u3305900 & u3276740)

Code is found in the files `puzzle_game.py` and `priority_queue.py` separated due to having different authors/programs so work could be done at the same time.

- **Priority event queue system** (Coded by u3305900)
 - Code is based on a variation of week 12 tutorial task3_2, this one was a lot easier as it did not need to buttons that caused issues in the heap visualizer, it is set up to be done in 12 hour time, the code will work on other lists but the data will need to be added to both the 'times' list and the 'events' dictionary.
- **Pathfinding puzzle** (Coded by u327674)
 - The `def pathfinding_visualise` works by using the `def draw_grid` helper function to draw a grid that the user can see and manipulate by clicking on squares to turn them into start, end, or obstacle. Then once the grid and its obstacles are set clicking the start button, `def dijkstra` is called which calls `def get_neighbors` which records a valid move and its cost, which `def dijkstra` adds to a queue and then finds the lowest cost queue. Once a path is found it is given to `def trace_path` which switches the grid's colours to create an animated effect.
- **Dynamic programming puzzle** (Coded by u327674)
 - The `def dynamic_programming_visualiser` works by reusing the `def draw_grid` helper function to draw a grid that the user can see and manipulate by clicking on to create obstacles. With the `def dynamic_programming_visualiser` handling the core dynamic logic by creating an overlapping grid which displayed the number of possible paths to each square, with the `def reconstruct_path` simply using the stored paths to animate one of the possible paths for the user to see.